

My Azure Hub-and-Spoke Lab

What I Built

This was my first real Azure networking lab and honestly, it was a bit of a headache at first.

The idea is simple: you have a central "**hub**" network and two "**spoke**" networks that only talk to each other **THROUGH** the hub. Not directly. Never directly.

I kept trying to ping `spoke2` from `spoke1` and it just would not work. It took me like an hour to realize Azure VNet peering does not work like a normal router. It is **non-transitive**, which means even if Hub talks to Spoke 1 and Hub talks to Spoke 2, Spoke 1 and Spoke 2 still cannot see each other. You have to force the traffic through something in the hub that knows how to forward it.

So, I put a Linux VM in the hub, turned on IP forwarding at both the Azure NIC level and inside the OS, and suddenly ping worked. Felt pretty good.

The Architecture

What is in the Diagram

Resource Name	Type / Role	Description / What it does
<code>vnet-hub-dev-ukw-001</code>	Hub Virtual Network	The hub network (<code>10.1.0.0/16</code>). Contains <code>subnet1</code> (<code>10.1.1.0/24</code>) where the routing VM resides.
<code>vnet-spoke1-dev-ukw-001</code>	Spoke 1 Virtual Network	Spoke 1 network (<code>10.2.0.0/16</code>). Isolated workload space containing <code>subnet1A</code> .
<code>vnet-spoke2-dev-ukw-001</code>	Spoke 2 Virtual Network	Spoke 2 network (<code>10.3.0.0/16</code>). Another isolated workload space containing <code>subnet1B</code> .
<code>vm-hub-fw-dev-ukw-001</code>	Virtual Machine	Linux firewall/routing VM (Ubuntu 22.04 LTS, size <code>Standard_B1s</code>).
<code>vm-hub-fw-dev-ukw-001-ip</code>	Public IP Address	Public IP assigned to the hub VM so I can SSH into it from home.
<code>vm-hub-fw-dev-ukw-001-nic</code>	Network Interface	The virtual network interface card (NIC) for the hub VM with IP forwarding enabled.
<code>vm-hub_key</code>	SSH Key Pair	Secure Shell credentials downloaded locally when initializing the VM.
<code>nsg-spoke1-dev-ukw-001</code>	Network Security Group	Security rules guarding Spoke 1. Only allows inbound SSH from my public IP and ICMP (ping) from Spoke 2.
<code>rt-spoke-dev-ukw-001</code>	Route Table	Custom routing rules. Overrides default routing to direct Spoke-to-Spoke traffic to the hub VM.

NetworkWatcher_ukwest

Network
Watcher

An automatic regional diagnostic resource created by Azure to monitor and troubleshoot network issues.

How I Built It

1. **Resource Group First:** Created `rg-hub-spoke-dev-ukw-001` in the `UK West` region. Putting everything in one group makes clean-up extremely easy.
2. **Hub VNet:** Built `vnet-hub-dev-ukw-001` with an address space of `10.1.0.0/16` and added `subnet1 (10.1.1.0/24)`.
3. **Spoke VNets:** Built `vnet-spoke1-dev-ukw-001 (10.2.0.0/16)` and `vnet-spoke2-dev-ukw-001 (10.3.0.0/16)` to avoid overlapping IP spaces. Created `subnet1A` and `subnet1B` inside them.
4. **Hub VM:** Deployed `vm-hub-fw-dev-ukw-001` (Ubuntu 22.04 LTS, `Standard_B1s`) into `subnet1`. Configured a Public IP for administrative access.
5. **Enable IP Forwarding:** This must be enabled in **TWO** places:
 - **Azure NIC Level:** In the Azure Portal, go to `VM > Networking > NIC > IP configurations` and toggle *IP forwarding* to **Enabled**.
 - **Inside the OS:** SSH into the Linux VM and enable IP forwarding in the kernel:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

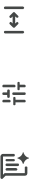
To make this configuration permanent across reboots, run:

```
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
```

If you skip either step, the VM will silently drop any transit packets.

6. **VNet Peerings:** Set up peerings between Hub ↔ Spoke 1 and Hub ↔ Spoke 2. You must check **"Allow forwarded traffic"** on **BOTH** sides of **BOTH** peerings (4 options to check in total).
7. **Route Tables:** Created `rt-spoke-dev-ukw-001` with a user-defined route directing Spoke 2 traffic to the Hub VM's private IP. Associated it with `subnet1A`. Created a reciprocal route table for Spoke 2 pointing to Spoke 1 and associated it with `subnet1B`.
8. **Network Security Groups (NSGs):** Configured `nsg-spoke1-dev-ukw-001` to restrict SSH access exclusively to my personal external IP address, and added an inbound security rule allowing ICMP traffic from Spoke 2's subnet range.
9. **Test VMs:** Provisioned `vm-spoke1-test-dev-ukw-001` in `subnet1A` and `vm-spoke2-test-dev-ukw-001` in `subnet1B` to serve as endpoints.
10. **The Verification Test:** SSH'd into Spoke 1 and initiated a ICMP request directly to Spoke 2's private IP address:

```
ping -c 4 <spoke2-private-ip>
```



Received a 100% success rate (0% packet loss). Executed a `traceroute` to verify that traffic hopped through the Hub VM router rather than taking a direct route.

What I Got Wrong

- **Missing Peering Settings:** I forgot to select the "Allow forwarded traffic" checkbox when creating VNet peerings, leading to silent packet drops.
- **Kernel Routing Off:** I enabled IP forwarding on the Azure NIC interface but forgot to configure the `sysctl` properties on the Linux kernel.
- **Orphaned Route Tables:** Built the route table but forgot to explicitly bind it to the spoke subnets.
- **Dynamic IP Rotation:** My ISP rotated my residential public IP, instantly blocking my SSH access until I updated the corresponding NSG inbound rules.

What I Learned

- Azure Virtual Network peering is strictly non-transitive. It prevents implicit traversal to improve perimeter security boundaries.
- Transit routing with custom appliances requires identical permission configurations at the SDN infrastructure layer and local host operating system kernel.
- Route tables do not impact network traffic unless they are actively associated with a target subnet.
- Document local interface parameters and internal private IPs; they are tedious to extract individually when configuring multi-subnet route tables later.
- `terraform-docs` dynamically maintains readmes when changing configuration inputs and outputs, eliminating outdated environment logs.

Tech Stack

- **Cloud Infrastructure:** Microsoft Azure Portal & Azure CLI
- **Infrastructure as Code:** HashiCorp Terraform & `terraform-docs`
- **Operating Systems:** Ubuntu 22.04 LTS
- **Diagnostic Utilities:** SSH, `ping`, `traceroute`

Workspace Files

- `hub-spoke-peering.png` - Visual topology rendering.
- `ARCHITECTURE.md` - Technical layout specifications (this file).
- `README.md` - Main repository guide automatically formatted with automated variables.

Created in June 2026 for the Azure Networking & IaC evaluation lab. Active resources were decommissioned following success metrics to avoid unnecessary consumption.